AWK Cheat Sheet - Compact & Practical

---

1. Basic Structure

```
awk 'pattern { action }' file
```

- `pattern`: optional, which lines to match
- `action`: optional, what to do with matching lines
- Default behavior: if no pattern, applies to all lines; if no action, prints matching lines
- Often used with pipes: `command | awk '...'`
- Use `exit` to stop processing input and exit `awk` with a specific status.

---

2. Printing

Action	Example
Print whole line	`awk '{ print $0 }' file`
Print specific columns	`awk '{ print $1, $3 }' file`
Print line number	`awk '{ print NR, $0 }' file`
Print number of fields	`awk '{ print NF }' file`

- `$0` = full line, `$1`..`$NF` = fields, `NR` = line number, `NF` = number of fields

---

3. Field Separators

```
awk -F ':' '{ print $1 }' /etc/passwd
awk 'BEGIN { FS=":"; OFS="-" } { print $1, $2 }' file
```

- `FS` = input field separator (default: space/tab)
- `OFS` = output field separator
- `BEGIN` block executes before processing any lines (setup, headers)
- `END` block executes after all lines are processed (summaries, totals)

---

4. Patterns & Conditions

```
awk '/error/ { print $0 }' file          # match regex
awk '$3>100 { print $1, $3 }' file       # numeric condition
awk 'NR>2 { print $0 }' file             # skip first 2 lines
awk 'NR==5 { print $0 }' file            # print specific line
```

---

5. Arithmetic

```
awk '{ sum+=$1 } END { print sum }' file     # sum first column
awk '{ print $1+$2, $2*2 }' file           # arithmetic per line
awk '{ product*=$2 } END { print product }' file
```

---

6. Strings & Text

```
awk '{ print length($0) }' file               # line length
awk '{ print substr($1,1,3) }' file           # substring
awk '{ gsub(/old/, "new"); print }' file    # replace text
awk '{ if ($2 ~ /pattern/) print $1 }' file  # regex match in column
```

---

7. Conditional Logic

```
awk '{ if ($3>50) print $1; else print $2 }' file
awk 'BEGIN { print "Start" } END { print "Done" }'
```

- Supports `if`, `else if`, `else` blocks
- Use `BEGIN` for setup/headers and `END` for summaries/totals

---

8. Logical Operators

Operator	Meaning	Example
`&&`	AND (both true)	`$2!="" && $3>50 { print $1 }`
`||`	OR (either true)	`$2=="" || $3<10 { print $1 }`
`!`	NOT (negate)	`!($2=="") { print $2 }`
`==`	Equal	`$1=="root"`
`!=`	Not equal	`$1!="root"`
`<`	Less than	`$3 < 100`
`>`	Greater than	`$3 > 50`
`<=`	Less or equal	`$4 <= 200`
`>=`	Greater or equal	`$4 >= 200`

Example Usage

```
awk '$2!="" && $3>50 { print $1, $2, $3 }' file   # AND
awk '$2=="" || $3<10 { print $1, $2, $3 }' file   # OR
awk '!($2=="") { print $2 }' file                 # NOT
```

---

9. Ternary Operator (Default values)

```
awk '{ print ($2=="" ? "-" : $2) }' file       # single column
```

```
awk '{ print ($2=="" ? "-" : $2), ($3=="" ? "-" : $3), ($4=="" ? "-" : $4) }' file  # multiple columns
```

- `$2=="" ? "-" : $2` → if `$2` empty, print `-`, else `$2`
- **Shorter (and often preferred):** `$2 ? $2 : "-"`
- Handles missing or empty columns in CSV/whitespace data

With Logic

```
awk '{ print ($2=="" || $3<10 ? "-" : $2) }' file
```

- Combines logic with default values and logical operators (AND/OR)

---

10. Counting / Unique Values

```
awk '{ counts[$1]++ } END { for(i in counts) print i, counts[i] }' file
```

- Count occurrences of column values (using `counts` as an associative array)
- Use `END` block for summary after processing all lines

---

11. Pass Shell Variables

```
awk -v var=100 '$3>var { print $1 }' file
```

- `-v var=value` passes shell variables into AWK

---

12. Loops

```
awk '{ for(i=1;i<=NF;i++) sum+=$i } END { print sum }' file
```

- Loop over fields for calculations
- Can combine with logical operators and ternary operator

---

13. Formatted Printing with printf

```
awk '{ printf "%s: %d\n", $1, $2 }' file
```

- `printf` allows precise formatting (like C)
- Format specifiers: `%s` = string, `%d` = integer, `%f` = float
- Can align columns, set width, control decimal places

Example: Table with Headers

```
awk 'BEGIN { printf "% -10s % -8s % -5s\n", "Name", "Age", "Score"; print "-----------------------" }
```

```
{ printf "% -10s % -8s % -5d\n", $1, $2, $3 }' file.txt
```

- `BEGIN` prints table header before processing lines
- `%-10s` = left-justified string width 10, `%-5d` = left-justified integer width 5
- Produces a neatly aligned table

---

14. Examples

```
# Print IP and MAC from netdiscover
awk 'NF>0 && !/Active scan/ { print $1, $2 }' netdiscover.txt

# Replace 'foo' with 'bar'
awk '{ gsub(/foo/, "bar"); print }' file.txt

# Conditional default value with logic
awk '{ print ($2 ? $2 : "-"), ($3 ? $3 : "-") }' file.txt

# Conditional AND / OR
awk '$2!="" && $3>50 { print $1, $2, $3 }' file
awk '$2=="" || $3<10 { print $1, $2, $3 }' file
```

- `BEGIN` for initialization/headers
- `END` for totals, summaries, cleanup
- Logical operators (`&&`, `||`, `!`) and ternary (`? :`) allow compact conditional expressions
- `printf` for formatted output and aligned tables

---

15. Built-in Variables

Variable	Description
`NR`	Number of the current record (line number)
`NF`	Number of fields in the current record
`FS`	Input field separator (default: space/tab)
`OFS`	Output field separator (default: space)
`RS`	Input record separator (default: newline)
`ORS`	Output record separator (default: newline)
`FILENAME`	Name of the current input file

---

16. Common Functions

Function	Description	Example
`length(s)`	Returns the length of string `s`.	`length($0)`
`substr(s, p, n)`	Returns substring of `s` starting at `p` for `n` chars.	`substr($1, 1, 3)`
`index(s,`	Returns the first position of	

`t)`	string `t` in `s`.	`index("hello", "ell")`
`match(s, r)`	Returns the position where regex `r` matches `s`.	`match($0, /pattern/)`
`split(s, a, fs)`	Splits string `s` into array `a` by `fs`.	`split("a,b,c", arr, ",")`
`gsub(r, t, s)`	Global substitution of regex `r` with `t` in `s`.	`gsub(/old/, "new", $0)`
`sub(r, t, s)`	First substitution of regex `r` with `t` in `s`.	`sub(/old/, "new", $0)`
`tolower(s)`	Converts string `s` to lowercase.	`tolower($1)`
`toupper(s)`	Converts string `s` to uppercase.	`toupper($1)`